

TREE 1

PROBLEM 9

QUESTION:

DS	Session	Tree 1	Question Information	Level 1	Challenge 69
Question description A forest is an undirected graph without cycles (not necessarily connected). Mohana and john are friends in kerala, both of them have a forest with nodes numbered from 1 to n, and they would like to add edges to their forests such that: • After adding edges, both of their graphs are still forests. • They add the same edges. That is, if an edge (u,v) is added to Mohana's forest, then an edge (u,v) is added to john's forest, and vice versa. Mohana and johns want to know the maximum number of edges they can add, and which edges to add. Constraints: $1 \leq n \leq 105,$ $0 \leq m1$ $m2 < n$ $1 \leq u, v \leq n, u \neq v$ Input The first line contains three integers n, $m1$ and $m2$ — the number of nodes and the number of initial edges in Mohana's forest and john's forest. Each of the next $m1$ lines contains two integers u and v — the edges in Mohana's forest. Each of the next $m2$ lines contains two integers u and v ($1 \leq u, v \leq n, u \neq v$) — the edges in Johns's forest. Output The first line contains only one integer k, the maximum number of edges Mohana and John can add. Each of the next k lines contains two integers u and v ($1 \leq u, v \leq n, u \neq v$) — the edge you add each time. If there are multiple correct answers, you can print any one of them.					
✓ Logical Test Cases Test Case 1 INPUT (STDIN) <pre>5 3 2 5 4 2 1 4 3 4 3 1 4</pre> EXPECTED OUTPUT <pre>1 1 5</pre> Test Case 2 INPUT (STDIN) <pre>5 3 2 5 3 2 3 2 3 2 3 2 4</pre> EXPECTED OUTPUT <pre>1 1 2 4 5</pre>					
✓ Mandatory Test Cases Test Case 1 KEYWORD <pre>while(m1--)</pre>					

▼ Complexity Test Cases

Test Case 1	Test Case 2	Test Case 3
CYCLOMATIC COMPLEXITY	TOKEN COUNT	NLOC
2	500	30

SOURCE CODE:

```
#include <stdio.h>

typedef struct DisjointSet {
    int leader[1005];
} DisjointSet;

int get_root(DisjointSet *ds, int x) {
    if (ds->leader[x] == x) return x;
    return ds->leader[x] = get_root(ds, ds->leader[x]);
}

int merge_sets(DisjointSet *ds, int x, int y) {
    int root_x = get_root(ds, x);
    int root_y = get_root(ds, y);
    if (root_x == root_y) return 0;
    ds->leader[root_x] = root_y;
    return 1;
}

int main() {
    int vertex_count, edges_a, edges_b;
    if (scanf("%d %d %d", &vertex_count, &edges_a, &edges_b) != 3) return 0;

    DisjointSet forest_a, forest_b;
    for (int i = 1; i <= vertex_count; i++) {
        forest_a.leader[i] = i;
        forest_b.leader[i] = i;
    }

    for (int i = 0; i < edges_a; i++) {
        int u, v;
        scanf("%d %d", &u, &v);
        merge_sets(&forest_a, u, v);
    }

    for (int i = 0; i < edges_b; i++) {
        int u, v;
        scanf("%d %d", &u, &v);
        merge_sets(&forest_b, u, v);
    }

    int new_u[1005], new_v[1005];
    int added_count = 0;

    for (int i = 1; i <= vertex_count; i++) {
        for (int j = i + 1; j <= vertex_count; j++) {
            if (get_root(&forest_a, i) != get_root(&forest_a, j) &&
                get_root(&forest_b, i) != get_root(&forest_b, j)) {
                merge_sets(&forest_a, i, j);
                merge_sets(&forest_b, i, j);
                new_u[added_count] = i;
                new_v[added_count] = j;
                added_count++;
            }
        }
    }

    printf("%d\n", added_count);
    for (int i = 0; i < added_count; i++) {
        printf("%d %d\n", new_u[i], new_v[i]);
    }
    return 0;
}
```

OUTPUT:

```
5 3 2
5 4
5 3
2 1
4 3
2 3
1
1 5
```

```
5 3 2
5 3
2 3
2 3
2 3
2 4
1
1 2
4 5
```